

Lineamientos para la

Descripción del Pipeline de Datos

Entregable 4 · Práctica de Arquitectura y Diseño de Sistemas 2026

Este documento define los lineamientos, la plantilla y ejemplos completos para describir el pipeline de datos de su sistema. Un pipeline de datos documenta cómo los datos fluyen a través del sistema: desde su origen, pasando por transformaciones, hasta el almacenamiento de los resultados y su consumo. A este documento se le adjunta otro que contiene ejemplos de rellenado de la plantilla dada.

1. ¿Qué es un pipeline de datos?

Un pipeline de datos es una secuencia de pasos que procesan datos: los recolectan de una o más fuentes, los transforman o enriquecen, y los almacenan o envían a un destino. No todo sistema tiene un pipeline complejo — pero todo sistema que procesa datos tiene al menos uno implícito.

El requerimiento arquitectónico del proyecto pide incorporar algún mecanismo de procesamiento de datos: transformación, agregación, integración de fuentes externas, o procesamiento batch/streaming. El pipeline describe CÓMO se implementa ese mecanismo.

Ejemplos de pipelines según el dominio:

Sistema	Ejemplo de pipeline
Comparador de precios	Scraping de supermercados → normalización de datos → comparación → respuesta al usuario
Monitoreo marítimo	Sensor envía telemetría → ingestión MQTT → evaluación de umbrales → generación de alerta
Entregas de provisiones	Solicitud creada → validación de stock → aprobación → asignación de envío → notificación
Estacionamiento	Inicio de sesión → cálculo de tarifa → cobro → cierre → generación de métricas diarias

2. ¿Qué se evalúa?

La cátedra evalúa los criterios de Procesamiento e Integración de Datos. Para aprobarlo, el documento de pipeline debe demostrar:

Claridad sobre qué datos se procesan, en qué momento, y por qué componente.

El flujo de datos está alineado con los diagramas de arquitectura y el modelo de datos (entrega 3).

Se documenta el tipo de procesamiento: batch (periódico) vs streaming (en tiempo real).

Se especifica dónde se originan los datos, cómo se transforman, y dónde terminan.

Se mencionan decisiones relevantes: por qué se eligió ese enfoque de procesamiento.

3. Las 7 preguntas que debe responder el pipeline

Para cada pipeline del sistema, deben poder responder estas siete preguntas. La plantilla de la sección siguiente está organizada en torno a ellas.

Nº	Pregunta	Lo que se espera responder
¿QUÉ?	Qué datos se procesan	Entidades, campos, volumen aproximado, formato de entrada
¿CUÁNDO?	Cuándo se ejecuta	En tiempo real (streaming), periódicamente (batch), o ante un evento (event-driven)
¿DESDE?	De dónde vienen los datos	Fuente: base de datos, API externa, sensor, archivo, evento, mensaje en cola
¿QUIÉN?	Qué componente lo hace	Nombre del contenedor o servicio responsable de cada paso
¿QUÉ HACE?	Qué transformación ocurre	Filtrar, normalizar, agregar, enriquecer, validar, calcular
¿DÓNDE?	Dónde se guardan los resultados	Tabla, colección, caché, archivo, evento publicado, notificación
¿POR QUÉ?	Por qué este enfoque	Justificación de la estrategia elegida (conectar con el ADR si aplica)

4. Tipos de procesamiento

Deben identificar el tipo de procesamiento de su pipeline — un sistema puede tener más de uno.

Tipo	Descripción	Cuándo aplica
Streaming (tiempo real)	Los datos se procesan a medida que llegan, sin esperar a acumularlos.	Eventos frecuentes que requieren respuesta inmediata.
Batch (por lotes)	Los datos se acumulan y se procesan en bloque, típicamente en forma programada.	Datos históricos, reportes, métricas periódicas.
Event-driven (por eventos)	El procesamiento se dispara ante un evento de negocio, no por tiempo ni por volumen.	Cambios de estado, acciones del usuario.
ETL (Extract-Transform-Load)	Extracción de una fuente, transformación del formato/estructura, y carga en destino.	Integración de fuentes externas con formatos distintos.

5. Plantilla — copiar y completar

Completar una sección de este tipo por cada pipeline identificado en el sistema. Un sistema puede tener uno o varios pipelines distintos.

Un pipeline = un flujo de datos con un propósito de negocio concreto. Si tienen scraping de precios Y notificaciones de cambio de estado, son dos pipelines distintos.

PLANTILLA — Pipeline [Nombre descriptivo del pipeline]

Descripción de una oración: qué hace este pipeline y para qué sirve al negocio.

PASO 1 — ORIGEN DE LOS DATOS

¿Qué datos?	Descripción de los datos de entrada: entidades, campos clave, formato. Ej: 'Lecturas de sensores en formato JSON {valor, unidad, sensorId, timestamp}'
¿Desde dónde?	Fuente concreta: nombre de la tabla, endpoint de API externa, topic MQTT, evento publicado, archivo CSV, etc.
¿Con qué frecuencia llegan?	¿Cada cuánto tiempo o ante qué evento llegan los datos de entrada?
¿Volumen estimado?	Orden de magnitud: filas por día, mensajes por segundo, MB por ejecución, etc.



PASO 2 — DISPARADOR / TRIGGER

¿Cuándo se ejecuta?	Programado (cron), en tiempo real, ante un evento de negocio, o manualmente. Indicar la frecuencia o el evento.
¿Quién lo dispara?	El componente o actor que inicia la ejecución: scheduler, mensaje en cola, request HTTP, evento de BD.



PASO 3 — PROCESAMIENTO

¿Qué componente lo ejecuta?	Nombre del contenedor/servicio responsable. Tecnología utilizada (ej: Python Worker, Node.js Backend).
Paso 1 — [nombre]	Descripción concisa de qué hace este paso: filtrar, validar, enriquecer, transformar, calcular, normalizar.
Paso 2 — [nombre]	Descripción del paso siguiente. Agregar tantos pasos como necesiten.
Paso N — [nombre]	Último paso de transformación antes de la salida.
¿Hay lógica de error?	¿Qué pasa si un paso falla? ¿Se reintenta, se descarta, se registra el error?



PASO 4 — SALIDA / DESTINO

¿Qué se produce?	Descripción del output: registros persistidos, eventos publicados, notificaciones enviadas, archivos generados.
¿Dónde se guarda?	Destino concreto: nombre de tabla/colección, caché Redis, cola de mensajes, bucket S3, endpoint externo.
¿Quién consume el resultado?	El componente o actor que usará los datos procesados: API REST, frontend, otro pipeline, usuario final.



PASO 5 — DECISIÓN ARQUITECTÓNICA

¿Por qué este enfoque?	Justificación de la estrategia de procesamiento elegida. Conectar con el ADR correspondiente si existe.
Trade-offs aceptados	Qué se sacrifica con este enfoque (latencia, consistencia, complejidad, costo).

Si el pipeline tiene un ADR asociado, referenciar aquí: 'Ver ADR-XXX — [título del ADR]'

6. Checklist de entrega

Verificar que el documento de pipeline cumple todo lo siguiente:

- ¿Se identificaron todos los pipelines del sistema? (scraping, notificaciones, procesamiento de eventos, etc.)
- ¿Cada pipeline responde las 7 preguntas (qué, cuándo, desde dónde, quién, qué hace, dónde, por qué)?
- ¿Se especifica el tipo de procesamiento: streaming, batch o event-driven?
- ¿Los componentes mencionados coinciden con los del diagrama de contenedores?
- ¿Las entidades y tablas mencionadas coinciden con el modelo de datos de la entrega 3?
- ¿Se documentan los casos de error y la estrategia de manejo?
- ¿Se mencionan los trade-offs del enfoque elegido?
- ¿Se referencia el ADR correspondiente si existe?
- ¿El volumen estimado de datos es coherente con la estrategia de persistencia elegida?

Pregunta de autoevaluación: si otro equipo leyera solo el documento de pipeline, ¿podría implementar el flujo de datos sin hacer preguntas adicionales? Ese es el nivel de claridad que se espera.